Zero-Downtime Post-Quantum TLS 1.3 Migration: A Bridge-Server-Based Approach

Minjoo Sim[1 [0000−0001−5242−214X]], Subin Jo[1 [0009−0004−4352−8021]], Hyuntae Song[1 [0009−0008−0373−8701]], Eunseong Kim[1 [0009−0003−1015−5276]], and Hwajeong Seo[1 [0000−0002−2356−3055]]

Hansung University, Seoul 02876, Republic of Korea
{minjoos9797,chosubin1208, sht56790u, eunsungkim2005, hwajeong84}@gmail.com

Abstract. The rapid advancement of quantum computing threatens the security of widely deployed public-key cryptosystems, creating an urgent need for practical migration to post-quantum cryptographic (PQC) standards. Although the U.S. National Institute of Standards and Technology (NIST) and Korea's KpqC initiative have recently standardized PQC algorithms, integrating them into Transport Layer Security (TLS) 1.3 remains operationally challenging. Larger certificates, higher handshake costs, and incompatibility with legacy clients make naive deployment impractical in production environments. While hot reload has long been supported in classical TLS deployments (e.g., Nginx, HAProxy), these mechanisms were designed for RSA/ECC contexts with small keys and certificates, and they do not address PQC-specific challenges.
This work presents a systematic demonstration of *hot reload and rollback in a PQC-enabled TLS context*, incorporating a policy-driven state machine for staged migration and rollback under realistic constraints such as increased handshake latency and legacy-client compatibility. We propose a bridge-server-based framework that operates at the TLS library level, enabling zero-downtime migration across classical, hybrid, and PQC deployments. Experimental evaluation shows that PQC handshakes incur a ≈5.8×–6.4× increase in latency in compute-bound settings relative to ECC baselines, while the relative overhead is significantly smaller in network-bound scenarios, highlighting the importance of deployment context. These findings provide a feasible path toward secure and incremental PQC integration in TLS infrastructures, contributing to broader strategies for achieving crypto-agility under evolving cryptographic standards.

Keywords: Crypto Agility · Post-Quantum Cryptography · TLS 1.3 · Zero-Downtime Migration

1 Introduction

The emergence of large-scale quantum computers poses a profound threat to classical public-key cryptosystems. Shor's algorithm breaks schemes based on integer

factorization and discrete logarithms, including RSA [1], Diffie–Hellman [2], and ECC [3], while Grover's algorithm reduces the effective security of symmetric primitives (e.g., AES-k to $2^{k/2}$ operations) [4]. Together, these advances enable harvest now, decrypt later attacks, where adversaries collect encrypted traffic today with the intent of decrypting it once sufficient quantum resources become available.

To mitigate this threat, the U.S. National Institute of Standards and Technology (NIST) finalized its first set of post-quantum cryptographic (PQC) standards in 2024 [5], and the KpqC project established domestically ratified PQC standards in 2025 [6].

Transport Layer Security (TLS) 1.3, the foundation of secure Internet communication, separates key exchange and authentication from cipher suites, providing architectural flexibility [7]. In principle, this design facilitates PQC adoption without fundamental protocol changes. In practice, deployment is difficult due to larger certificates, higher handshake costs, and compatibility issues with legacy clients. These challenges highlight the need for a mechanism that integrates PQC without downtime, preserves interoperability, and supports rollback when failures occur.

Hot-reload mechanisms have long existed in classical TLS deployments (e.g., Nginx [8], HAProxy [9]). However, extending these techniques to post-quantum TLS is not straightforward. PQC certificates are substantially larger, handshake costs are higher, and interoperability with legacy clients must be preserved to avoid service disruption. These additional constraints make operational migration in the PQC context qualitatively more complex than in classical TLS.

Projects such as `liboqs` and `oqs-provider` extend TLS 1.3 by exposing standardized PQC key exchange and signature algorithms through the OpenSSL provider interface. These libraries enable experimentation with PQC handshakes and validation of algorithm-level interoperability. However, their scope is limited to cryptographic integration: they do not provide operational migration mechanisms such as downtime-free certificate rollover, staged rollout across heterogeneous clients, or rollback under failure. In production environments, these operational guarantees are as critical as algorithmic support. Thus, while `liboqs` and `oqs-provider` prove that PQC primitives can be embedded into TLS, they stop short of offering deployable migration strategies for real-world infrastructures.

We argue that a bridge-server approach is essential. By interposing a dedicated bridge between clients and servers, migration policies can be applied independently of application logic. This design enables staged rollout and rollback, supports heterogeneous clients, and ensures continuous availability during the transition to PQC. Unlike conventional TLS termination proxies, our bridge operates directly at the TLS library level, managing `SSL_CTX` instances and migration states rather than merely reloading configurations. This distinction is crucial to support rollback and staged PQC migration under live traffic.

In summary, TLS 1.3 deployments must be able to adopt PQC seamlessly while preserving compatibility with legacy clients. To address this requirement, we propose a bridge-server-based framework that supports policy-driven hot

reload and rollback, offering a practical path toward crypto-agility in the PQC era [10].

1.1 Contributions

This paper makes the following contributions:

- **Bridge-server-based framework:** We introduce a migration framework that goes beyond conventional proxies or load balancers. Whereas existing systems simply relay TLS traffic or reload certificates at the application layer, our bridge operates directly at the TLS library level, creating distinct `SSL_CTX` instances for each session and applying cryptographic policies according to the current migration state (`announce` → `prefer` → `require` → `rollback`). This design supports staged rollout and rollback under live traffic while addressing PQC-specific challenges—such as higher handshake latency and legacy-client compatibility—which prior hot-reload systems for classical TLS never considered. The practical significance lies in enabling **fine-grained session-level migration** and immediate rollback without process restart, providing operational guarantees that conventional proxy-level reload systems cannot offer.
- **Policy-driven state machine:** Unlike conventional reload mechanisms that immediately replace cryptographic material, our design introduces a staged state machine for controlled rollout and rollback. This mechanism provides fine-grained control of PQC adoption under live traffic, giving operators safety and flexibility unavailable in prior TLS systems.
- **Hot reload in a PQC context:** While prior hot-reload systems were designed for ECC/RSA environments, our approach adapts and evaluates hot reload and rollback mechanisms in post-quantum TLS. This contribution addresses PQC-specific challenges such as larger certificates, increased handshake latency, and legacy-client compatibility, thereby bridging the gap between algorithmic integration (e.g., liboqs) and operational deployment.
- **Experimental evaluation:** We validate the framework through controlled TLS experiments. Beyond a proof-of-concept, we systematically measure (i) handshake latency across 200 trials per configuration, (ii) InterQuartile Range (IQR) and 95% confidence intervals for stability, and (iii) migration time and correctness across staged states. The results confirm that continuous availability is preserved with moderate overhead relative to ECC baselines in compute-bound settings.

2 Background

2.1 TLS

TLS 1.3 establishes a secure channel with a single round-trip (1-RTT) handshake. The client initiates the connection with *ClientHello*, the server responds with *ServerHello*, and key agreement proceeds via the `supported_groups` and

`key_share` extensions. Once the handshake completes, application data are protected with symmetric encryption. Server authentication is provided through *Certificate* and *CertificateVerify*, and both parties exchange *Finished* messages to ensure transcript integrity [7].

In TLS 1.3, the client's *ClientHello* advertises its capabilities through fields such as `cipher_suites`, `supported_versions`, `supported_groups`, `key_share`, and `signature_algorithms`/`signature_algorithms_cert`. The server's *ServerHello* then communicates its selections.

An operationally important point is that in TLS 1.3 the cipher suite string specifies only the symmetric record-protection algorithm and the HKDF hash. Key establishment is determined by the negotiated group (`supported_groups`/`key_share`), while authentication depends on the certificate's signature algorithm (`signature_algorithms` and optionally `signature_algorithms_cert`). Thus, the cipher suite string alone cannot indicate PQC adoption: PQC KEMs appear in the negotiated group, and PQC authentication in the certificate's signature scheme—neither is visible in the cipher suite name. By contrast, TLS 1.2 cipher suite names bundled key exchange, authentication, and symmetric protection (e.g., TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256), making it possible to infer algorithms directly [11]. Because TLS 1.3 separates these planes, one can deploy a PQC KEM without altering the cipher suite, or adopt PQC authentication simply by changing the certificate's signature algorithm.

2.2 NIST PQC Standardization

The U.S. National Institute of Standards and Technology (NIST) conducted a multi-year standardization process [12]. In 2024, NIST finalized its first set of PQC standards—FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA) [13,14,15]—selecting ML-KEM (CRYSTALS-Kyber) [16] for key encapsulation and ML-DSA (CRYSTALS-Dilithium) for digital signatures, while SPHINCS+ (standardized as SLH-DSA) provides a stateless hash-based alternative [17]. Falcon remains a compact lattice-based signature scheme selected by NIST (FIPS forthcoming) [18]. More recently, NIST added HQC as an additional code-based KEM [19], further diversifying the standardized portfolio.

Security strength in PQC is defined in levels corresponding to classical work factors, with Level 1, Level 3, and Level 5 approximating AES-128, AES-192, and AES-256 key search complexities, respectively. Parameter sets are chosen in practice to achieve these levels while balancing efficiency, bandwidth, and implementation constraints. Table 1 summarizes the parameter sizes of the standardized NIST algorithms.

2.3 KpqC

South Korea launched the National Korean Post-Quantum Cryptography Competition (KpqC) in 2022 with the goal of establishing domestic cryptographic standards [6]. In January 2025, the final KpqC standards were announced. Two KEMs (NTRU+ [20] and SMAUG-T [21]) and two digital signature schemes

Table 1: Parameter sizes (in bytes) of standardized NIST PQC algorithms.

Type	Algorithm	Security Lvl.	Public Key	Secret Key	Ciphertext/Sig.
KEM	ML-KEM-512	1	800	1,632	768
	ML-KEM-768	3	1,184	2,400	1,088
	ML-KEM-1024	5	1,568	3,168	1,568
KEM	HQC-128	1	2,249	56	4,497
	HQC-192	3	4,522	64	9,042
	HQC-256	5	7,245	72	14,485
Signature	ML-DSA-44	2	1,312	2,528	2,420
	ML-DSA-65	3	1,952	4,000	3,293
	ML-DSA-87	5	2,592	4,864	4,595
Signature	Falcon-512	1	897	1,281	666
	Falcon-1024	5	1,793	2,305	1,280
Signature	SPHINCS+128s	1	32	64	7,856
	SPHINCS+128f	1	32	64	17,088
	SPHINCS+192s	3	48	96	16,224
	SPHINCS+192f	3	48	96	35,664
	SPHINCS+256s	5	64	128	29,792
	SPHINCS+256f	5	64	128	49,856

(AIMer [22] and HAETAE [23]) were standardized after a national evaluation process to meet domestic security policy requirements. Table 2 summarizes the parameter sizes of these algorithms.

2.4 Crypto Agility

Crypto agility denotes the capability to change cryptographic algorithms and parameters at runtime *via policy rather than code*, enabling safe migration under live traffic [24]. In TLS 1.3 this is operationally natural because record protection (cipher suite) is orthogonal to both key establishment (negotiated via supported_groups/key_share) and authentication (negotiated via signature_algorithms/signature_algorithms_cert) [7]. Hence, deployments can introduce a PQC KEM without changing the cipher suite, or adopt PQC authentication by switching the certificate's signature algorithm—both by updating policy rather than redeploying binaries.

2.4.1 Crypto Agility in TLS 1.3 In TLS 1.3, cipher-suite names enumerate only symmetric record protection and the hash, so neither the KEM nor the certificate-signature choice appears in the suite string; evidence of PQC use relies on the negotiated group and the certificate's signature algorithm. This conceptual decoupling enables migrations aligned with ongoing standards (e.g.,

Table 2: Parameter sizes (in bytes) of standardized KpqC algorithms.

Type	Algorithm	Security Lvl.	Public Key	Secret Key	Ciphertext/Sig.
KEM	NTRU+KEM576	1	864	1,760	864
	NTRU+KEM768	3	1,152	2,336	1,152
	NTRU+KEM864	3	1,296	2,624	1,296
	NTRU+KEM1152	5	1,728	3,488	1,728
KEM	SMAUG_T1	1	672	832	672
	SMAUG_T3	3	1,088	1,312	992
	SMAUG_T5	5	1,440	1,728	1,376
Signature	AIMer_128s	1	32	48	4,160
	AIMer_128f	1	32	48	5,888
	AIMer_192s	3	48	72	9,120
	AIMer_192f	3	48	72	13,056
	AIMer_256s	5	64	96	17,056
	AIMer_256f	5	64	96	25,120
Signature	HAETAE2	2	992	1,408	1,474
	HAETAE3	3	1,472	2,112	2,349
	HAETAE5	5	2,080	2,752	2,948

FIPS 203/204/205 and the NIST PQC project) and with national profiles (e.g., KpqC), without service interruption [13,14,15,12,25].

2.4.2 Related Works Existing systems already explore aspects of cryptographic agility across multiple layers. For example, Kubernetes [26] supports zero-downtime rolling updates of applications, while Envoy proxy [27] applies dynamic configuration updates via the xDS API. Cloudflare's PQC experiments [28] integrate quantum-resistant algorithms into TLS 1.3, but only as controlled production trials. At the web server layer, Nginx [8] supports hot reload of SSL configurations by restarting worker processes. Although these systems differ in scope, they collectively demonstrate that state-preserving reconfiguration is an established principle for achieving service continuity.

In contrast, our work focuses on real-time replacement of cryptographic algorithms *inside the TLS protocol stack* rather than configuration reload at the application or proxy level. This distinction enables capabilities that conventional proxies lack: (i) a policy-driven state machine (announce → prefer → require → rollback) for staged migration and safe rollback under failure, (ii) direct manipulation of TLS library contexts (`SSL_CTX`) instead of process-level reloads, and (iii) explicit support for PQC-specific challenges such as large certificates, higher handshake latency, and legacy-client fallback. Together, these features extend hot reload from simple configuration management to true crypto-agility in the TLS handshake.

To make this contrast clear, Table 3 summarizes the capabilities of representative systems—Nginx, Envoy, and Cloudflare— alongside our proposed ap-

proach, highlighting our unique support for rollback, policy-based migration, and full PQC integration.

Unlike Nginx or Envoy, which reload configurations at the process level and apply a single global policy to all new sessions, our bridge operates directly on `SSL_CTX` instances inside the TLS library. This enables **per-session policy enforcement**, allowing some clients to remain on ECC while others migrate to hybrid or PQC during the same deployment cycle.

Table 3 further highlights this distinction. Whereas existing hot-reload mechanisms lack rollback and staged migration support, our bridge combines per-session `SSL_CTX` selection with a policy-driven state machine, transforming hot reload from simple configuration management into an operational migration strategy tailored for PQC.

Table 3: Comparison of Hot Reload and PQC Support across Systems. Red checkmarks indicate features unique to our system, which are enabled by a state-machine-based migration design.

System / Project	Hot Reload	Downtime -free	Rollback	Policy-based Migration	PQC Support
Nginx [8]	✓	✓	✗	✗	✗
Nginx (with OQS build) [29]	✓	✓	✗	✗	✓*
Envoy [27]	✓	✓	✗	✗	✗
Cloudflare PQC [28]	Trial only	✓	✗	✗	✓†
Our System	✓	✓	✓	✓	✓

* Experimental support via OQS-enabled OpenSSL build.
† Limited beta-only deployment.

3 Proposed Method

3.1 System Overview and Objectives

Figure 1 illustrates the overall architecture of the proposed system. Figure 1 (a) shows the scenario when the client uses classic TLS, while Figure 1 (b) depicts the scenario when PQC is used. Unlike conventional TLS proxies that reload configuration files at the process level, the proposed bridge operates directly at the TLS library level by configuring `SSL_CTX` instances for each new handshake. This enables migration policies to be applied at session creation time, ensuring that the negotiated cryptography reflects the policy in effect at that moment. The system consists of two servers employing different cryptographic protocols and a bridge that routes client requests to the appropriate server. Policies are reloaded via the `SIGHUP` signal (Section 3.2.2). During this process, existing sessions continue under the previous configuration, while newly established handshakes immediately adopt the updated policy. Because `SSL_CTX` reconfiguration

is performed at runtime without restarting processes, the bridge achieves seamless policy updates without delay or downtime.

Listing 1.1 shows a simplified excerpt where the bridge creates a new `SSL_CTX` and applies PQC-capable groups and signature algorithms before the TLS handshake. This demonstrates that cryptographic policies are enforced at the library level rather than through external configuration reloads.

```
void *server_ctx = crypto_engine_create_ctx(&g_crypto_engine,
    1, 2);
SSL_CTX *ssl_ctx = (SSL_CTX*)server_ctx;

// Apply PQC key-exchange groups (runtime policy)
const char *pqc_groups =
  "smaug1:mlkem512:kyber768:ntrupluskem864:X25519:P-256";
SSL_CTX_set1_groups_list(ssl_ctx, pqc_groups);

// Apply PQC signature algorithms (runtime policy)
const char *pqc_sigalgs =
  "ecdsa_secp256r1_sha256";
SSL_CTX_set1_sigalgs_list(ssl_ctx, pqc_sigalgs);
```

Listing 1.1: Per-session `SSL_CTX` configuration at handshake creation.

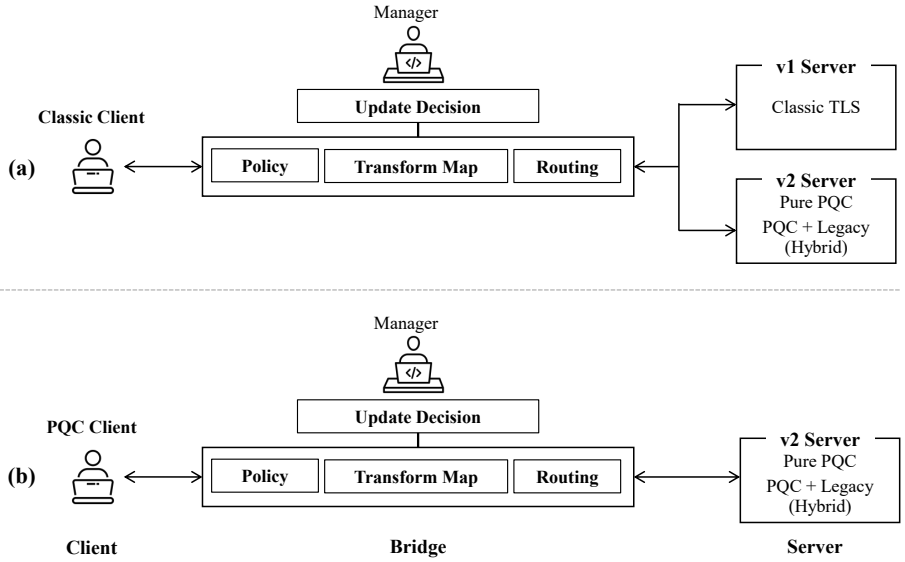

Fig. 1: System overview. Clients are routed to the appropriate server via the bridge according to their cryptographic protocol and the bridge's policy. (a): Classic Client, (b): PQC Client

3.2 Architecture

3.2.1 System Components The system consists of clients, a bridge, and servers. To support both legacy and modern PQC clients, it adopts an intermediary relay through the bridge. The bridge routes clients to appropriate servers based on client version.

Client The client is the entity requesting the service and is categorized into clients using PQC and those using classic TLS. During the TLS handshake process, the client transmits cryptographic information (see Section 2.1) to negotiate encryption capabilities through the bridge.

Bridge This bridge is designed to ensure zero downtime and zero delay even during critical updates such as protocol changes or minor updates like message modifications. It functions as a TLS-to-TLS relay between clients and game servers. Clients are routed through the bridge to the appropriate game server according to their version, based on policies defined by the administrator (See Section 3.3.2).

Unlike conventional proxies that reload a single global `SSL_CTX`, our bridge pre-initializes multiple `SSL_CTX` instances (classic, hybrid, and PQC) and selects among them on a per-session basis according to the migration policy state (Announce, Prefer, Require, Rollback). This library-level control enables staged rollout and rollback under live traffic without disrupting existing connections (see Listing 1.2).

```
/* Pre-initialize multiple SSL_CTX instances */
SSL_CTX *ctx_classic = init_ctx("cert_classic.pem");
SSL_CTX *ctx_hybrid  = init_ctx("cert_hybrid.pem");
SSL_CTX *ctx_pqc     = init_ctx("cert_pqc.pem");

/* Policy state machine for migration */
SSL_CTX *selected_ctx;
switch (g_policy.migration_mode) {
  case ANNOUNCE:
    selected_ctx = ctx_classic; break;
  case PREFER:
    selected_ctx = client_supports_pqc ? ctx_hybrid :
    ctx_classic; break;
  case REQUIRE:
    selected_ctx = ctx_pqc; break;
  case ROLLBACK:
    selected_ctx = ctx_classic; break;
}

/* Bind context per session */
SSL *ssl = SSL_new(selected_ctx);
if (SSL_accept(ssl) <= 0) {
    log_error("TLS␣handshake␣failed");
}
```

Listing 1.2: Policy-driven per-session selection of PQC-enabled `SSL_CTX` instances.

Servers Version 1 (v1) servers, which support classic TLS, and version 2 (v2) servers, which have PQC enabled including KpqC, coexist independently. The two communicate via a bridge using TLS. Clients are routed to the appropriate server through the bridge. The servers differ in their key exchange protocols: v1 is based on ECC [30], while v2 uses PQC and hybrid PQC key exchange methods. In this system, SMAUG-T KpqC is used as an example [21,31]. For compatibility, certificates and signatures are unified using RSA.

3.2.2 Transform Mechanism This system proposes a mechanism that hierarchically separates policies and enables independent changes to each layer through a hot reload approach, thereby realizing uninterrupted policy updates.

Hot-reload Hot Reload is a technology that immediately applies code changes during application runtime without stopping the app, allowing developers to quickly see the results while preserving the application's state[32,33,8]

The system employs a hot reload [32] mechanism that loads changed code and configurations into memory and rebuilds while preserving the program state, allowing immediate reflection of updates without terminating or fully restarting

the program. To achieve this, it leverages the SIGHUP signal, typically used to notify session termination or network disconnection [34], as a flag to reconfigure in-memory settings without killing the process for policy application.

Moreover, by structuring the bridge as a parent-child process model[1], policy changes in the parent process do not affect the child processes. Consequently, ongoing child process sessions maintain existing connections without interruption, while newly spawned child processes apply the updated policies.

Policy System The system clearly separates policy structures into three layers: security policies, routing and network connection management, and content transformation. Each layer operates independently, and higher-level policies determine the behavior of lower-level ones, preventing policy conflicts.

The security policy layer (*crypto_policy.ini*) governs security aspects, including PQC migration modes, encryption algorithms, and TLS versions. The routing policy layer (*routing.ini*) manages network connections such as server routing by client version, port mapping, and load balancing. The content transformation layer (*transform_map.tsv*) enhances user experience through real-time message transformation and UI text updates. This modular structure allows easy modification of new policies or formats, and each layer's policy is hot reloaded for interruption-free updates.

3.2.3 PQC Implementation The encryption policy supports not only classic TLS but also PQC and hybrid PQC protocols. This system can apply both NIST PQC and KpqC, utilizing the following libraries to enable their implementation.

CLOQS-Linux It is a PQC integrated library provided by `PQC Migration Platform`[2]. Built on top of the existing liboqs, it offers optimized implementations for various operating systems and low-power environments. Notably, while liboqs solely supports NIST PQC standard algorithms, CLOQS-Linux extends its support to encompass standard KpqC algorithms as well [35]. Table 4 lists the PQC algorithms supported by CLOQS-Linux. The presented system supports PQC and Hybrid PQC based on CLOQS-Linux.

oqs-provider It is a library designed to enable post-quantum cryptography in OpenSSL version 3.x and higher, allowing the activation of PQC KEM and signature algorithms during TLS communications [36,37]. However, the officially provided oqs-provider does not support KpqC algorithms. Therefore, in this system, similar to CLOQS-Linux, we utilized the KpqC-compatible oqs-provider provided by the `PQC Migration Platform`.

[1] The child process is created by duplicating the parent process, but it has a unique process ID and operates independently afterward.

[2] The first platform in South Korea to provide diverse PQC services. See `https://pqcmp.kr`.

Table 4: Supported PQC Algorithms in CLOQS-Linux. It includes KpqC, unlike the existing OQS library.

Type	**Algorithm**
NIST PQC	ML-KEM
	ML-DSA
	SLH-DSA (upcoming)
KpqC	SMAUG-T
	NTRU+
	HAETAE
	AIMer

3.3 Critical Update Scenario

Critical updates refer to major policy changes such as protocol modifications. In this section, we conducted a scenario of gradual migration from the v1 server (classic server) to the v2 server (PQC server).

3.3.1 Policy Criteria To ensure system stability and reliability, this study defines specific transition criteria. The evaluation follows the recommendations in the Google Site Reliability Engineering Workbook, where the monitoring windows are set as follows: a Long window of 1 hour and a Short window of 5 minutes. Transition rollback decisions are periodically evaluated based on the Long window, and if the defined threshold is not met, the system rolls back to the previous stage.

Three primary indicators—Client Compatibility, Latency, and Critical Error Rate—are used to evaluate the transition stability. Tables 5 and 6 summarize the quantitative criteria for both long-term and short-term rollback conditions, respectively.

Table 5: Long window (1 hour) transition evaluation and rollback conditions

Category	**Rollback Condition**	**Evaluation Criteria**
Client Compatibility	$S_{PQC} < S_{prefer} - 5\%$	PQC communication success rate compared to the *prefer* baseline
Latency	$R_{delay} > 5\%$	Average overhead ratio and proportion of abnormal delay
Critical Error Rate	$E_{fail} > 0.1\%$	Handshake failure, authentication, or encryption errors

Here, S_{PQC} denotes the PQC communication success rate, S_{prefer} represents the prefer baseline success rate, R_{delay} is the proportion of abnormal delays, and E_{fail} indicates the critical error rate.

Client compatibility is defined as the PQC communication success rate relative to the prefer ratio. In this study, the transition is considered stable if the success rate after migration does not decrease by more than 5% compared to the prefer ratio. For instance, if the prefer ratio is 25%, a transition is deemed stable when the actual communication success rate remains at or above 20

Latency is evaluated based on the measured overhead presented in this paper. The average overhead observed when transitioning from ECC to PQC is 5.81 times, which is used as the reference value. If the overhead exceeds 7 times, it is regarded as an abnormal delay. However, as long as such abnormal delays occur in less than 5% of total requests, the system transition is considered to be proceeding normally.

The critical error rate represents defects that directly affect client connectivity, such as handshake failures or authentication errors. If this error rate exceeds 0.1%, the system is considered to have encountered a severe failure, and rollback procedure is triggered.

In the event of a major system failure, an immediate rollback is determined based on the short window criteria.

For rapid detection and short-term mitigation of abnormal behavior, a Short window with a 5-minute period is used. This mechanism ensures immediate rollback when sudden performance degradation is observed. The short-term evaluation criteria are summarized in Table 6.

Table 6: Short window (5 minutes) immediate rollback conditions

Category	**Rollback Condition**	**Evaluation Criteria**
Client Compatibility	$S_{PQC} \geq 0.5 \times S_{prefer}$	PQC communication failure rate compared to prefer baseline
Latency	$R_{delay} \geq 30\%$	Proportion of delayed requests within the Short window
Critical Error Rate	$E_{fail} > 1\%$	Total connection failure rate including handshake errors

Here, S_{PQC} denotes the PQC failure ratio and E_{fail} represents the proportion of overall failed requests.

3.3.2 Policy-Based Routing The proposed system is designed with a routing policy that aims for a gradual transition to PQC servers (see Fig 2). By utilizing a weighted-based policy (prefer policy), clients are progressively directed to use PQC servers. In case of any issues, the routing environment can be rolled back at any time through policy control. This approach enables a smooth transition while maintaining service stability.

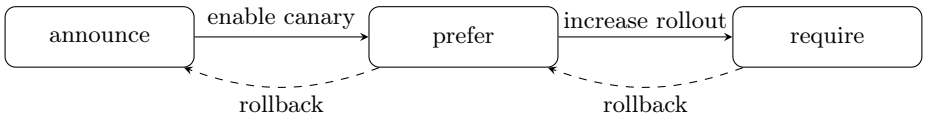

Fig. 2: State transition of routing policies enabling gradual PQC server migration for classical TLS clients, with rollback mechanisms ensuring service stability.

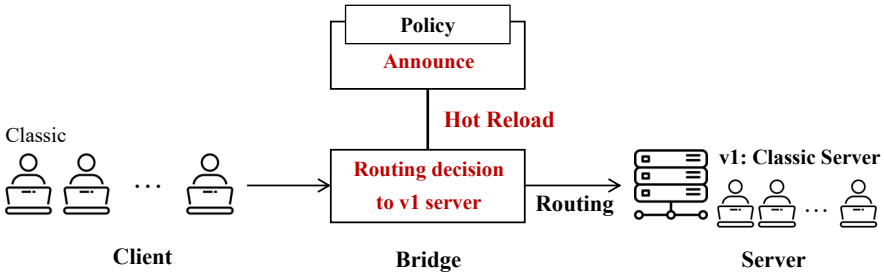

Fig. 3: Routing process in the **Announce** policy state. Clients using classic TLS are all routed to classic TLS server.

In the case of PQC clients, routing to PQC servers occurs immediately, regardless of the policy. Therefore, this section focuses on clients using classic TLS.

Announce Figure 3 illustrates the routing scenario of classic clients during the announce policy phase. In this phase, clients are informed that PQC servers are supported, but routing itself is conducted to the classic servers instead of the PQC servers.

Prefer Figure 4 illustrates the routing scenario for classic clients during the Prefer policy phase. The Prefer phase involves gradually routing classic clients initiating new handshakes to PQC servers. By adjusting the `rollout_percentage`, a portion of new handshakes is directed to PQC servers, enabling a staged rollout.

Require Figure 5 illustrates the routing scenario for classic clients during the Require policy phase. In this phase, the bridge enforces all new handshakes to route to PQC servers. However, ongoing sessions are not migrated.

Rollback If an error occurs, the Rollback policy can be executed to revert to the original policy.

3.4 Minor Update Scenario

Minor updates refer to light changes, such as simple text error corrections. This section covers the update scenario performed when simple text errors occur.

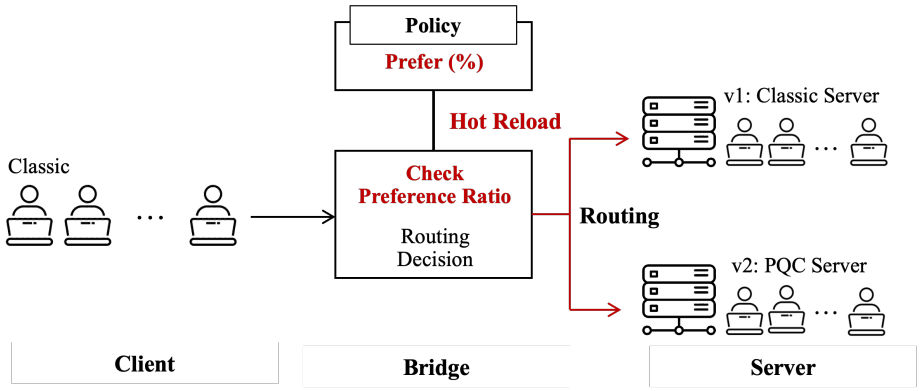

Fig. 4: Routing process in the **Prefer** policy state. classic TLS clients are routed to PQC server in proportion to the Prefer ratio. The higher the ratio, the greater the proportion of clients transitioning to PQC servers.

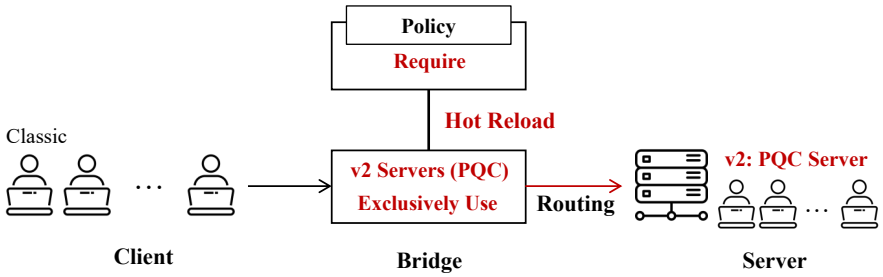

Fig. 5: Routing process in the **Require** policy state. Clients using classic TLS are also all routed to PQC servers.

Figure 6 illustrates the server update scenario in a minor update situation. When simple errors like typos are detected, administrators can resolve them by modifying the `Transform_map` without server downtime. The changes are not applied to ongoing sessions but take effect starting from new sessions.

4 Results

This section presents the results and discussion of testing a simple game server created based on the proposed system. In particular, it addresses performance changes resulting from policy modifications in the critical and minor scenarios proposed in Section 3, as well as differences in handshake performance according to the TLS protocol.

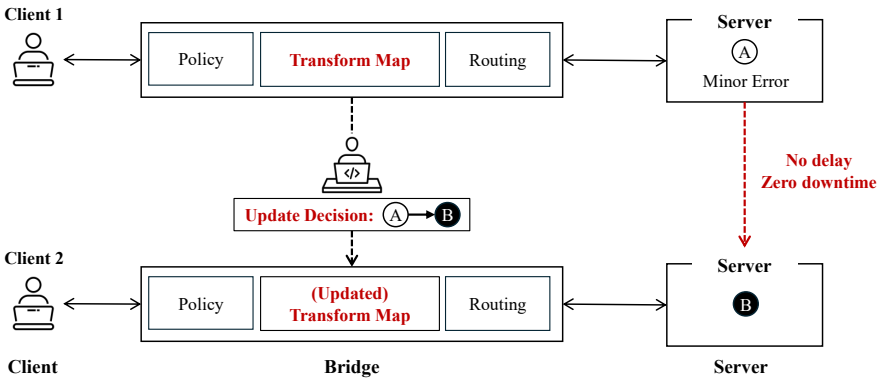

Fig. 6: Minor error correction using the Transform Map allows administrators to update mapping rules without downtime, applying changes immediately to new sessions while ongoing sessions remain unaffected.

4.1 Experimental Setup

Table 7 was built based on version Ubuntu 22.04 in two virtual environments to match the library environment provided by the PQC migration platform used in the study. Additionally, protocol communication tests using PQC-based KEM were conducted by utilizing OpenSSL and liboqs. The build supports cross-platform compilation using CMake 3.22.1 and compiles according to the GCC 11.4.0 standard.

Table 7: Experimental environments (single-host, two-terminal setup).

Label	OS	GCC	Virtual Env.	OpenSSL
Virtual 1	Ubuntu 22.04.5 LTS	11.4.0	VirtualBox	3.0.2
Virtual 2	Ubuntu 22.04.5 LTS	11.4.0	VMWare	3.0.2

4.2 Handshake Evaluation

The handshake performance was precisely analyzed through 200 repeated measurements of latency. Each measurement recorded the response delay during the individual handshake process in milliseconds (ms), and the interquartile range (IQR) was additionally calculated to evaluate the distribution and stability of the data.

4.2.1 Classic TLS Since the TLS 1.3 standard recommends ECC-based key exchange[7], this study evaluated the performance of Classic TLS using ECC-based key exchange. Table 8 shows the handshake performance of Classic TLS.

Table 8: TLS 1.3 handshake latency and IQR with ECC, measured over 200 repeated handshake attempts.

Type	Algorithm	Latency (ms)	IQR (ms)	95% CI High	95% CI Low
ECC	X25519	86.1	11.7	145.6	73.0
	X448	88.1	11.8	159.2	75.2
	P-256	85.8	11.7	161.2	70.9
	P-384	116.1	42.7	209.5	79.8
	P-521	90.7	10.3	183.4	76.7

4.2.2 PQC Handshake Table 9 shows the handshake performance of PQC. In this case, hybrid algorithms including NTRU+, except for the 576-bit key length, failed to produce valid measurements. This is likely due to compatibility issues and overload caused by the key length, which prevented the proper completion of the handshake process.

The measured PQC handshake performance showed a relatively high delay, averaging in the upper 600 ms range. This is interpreted as due to the inherently higher computational complexity and increased key length of PQC operations compared to classical TLS operations. However, this delay is an essential trade-off to achieve quantum-resistant security, providing long-term stability in future quantum computing environments. Furthermore, considering that the experiment was conducted in a virtual environment and the potential introduction of hardware acceleration, there is significant room for performance improvement through optimization.

Table 10 compares PQC and Hybrid KEMs against their corresponding ECC baselines, while Table 8 and Table 9 summarize handshake latency across ECC, PQC-only, and Hybrid KEMs in compute-bound experiments. The results show that PQC-only algorithms introduce a 5.8–6.4× increase relative to their ECC counterparts, with Hybrid KEMs consistently incurring slightly higher costs. Some Hybrid variants (e.g., NTRU+KEM-768/864/1152) were omitted due to implementation limitations.

4.3 Zero-Downtime Migration Evaluation

4.3.1 Critical Update Scenario: Migration to PQC Protocol We evaluated the performance of the PQC migration process by measuring migration time, success rate, and resource usage at each stage of the system. Ultimately, we measured the results of continuously performing incremental migrations (Announce → Prefer 25% → Prefer 50% → Prefer 75% → Require) to

Table 9: TLS 1.3 handshake latency and interquartile range (IQR) with PQC and hybrid key encapsulation mechanisms (KEMs) were measured over 200 repeated handshake attempts for each algorithm to ensure statistical robustness, with 95% confidence intervals (CI) for upper and lower bounds also calculated.

Type	PQC Algorithm	Hybrid(ECC)	Latency(ms)	IQR(ms)	CI High	CI Low
PQC-only	ML-KEM-512		536	4.6	537	536
	ML-KEM-768		535	3.8	536	535
	ML-KEM-1024		539	7.3	540	538
	SMAUG-T1		536	5.2	537	536
	SMAUG-T3		537	5.2	537	536
	SMAUG-T5		538	5.4	539	537
	NTRU+KEM-576		555	15.2	558	553
	NTRU+KEM-768		539	5.6	540	537
	NTRU+KEM-864		543	5.6	543	542
	NTRU+KEM-1152		542	9.2	544	540
Hybrid PQC	ML-KEM-512	secp256r1	557	6.6	559	555
	ML-KEM-768	secp384r1	557	7.2	559	555
	ML-KEM-1024	secp521r1	554	7.5	555	553
	SMAUG-T1	secp256r1	551	5.4	552	551
	SMAUG-T3	secp384r1	561	10.8	564	559
	SMAUG-T5	secp521r1	553	6.2	554	551
	NTRU+KEM-576	secp256r1	555	7.5	556	553
	NTRU+KEM-768	secp384r1	–	–	–	–
	NTRU+KEM-864	secp384r1	–	–	–	–
	NTRU+KEM-1152	secp521r1	–	–	–	–

Table 10: Handshake latency of PQC and Hybrid KEMs relative to their ECC counterparts (compute-bound).

Type	Algorithm	ECC Baseline	Overhead vs. ECC
Hybrid	ML-KEM-512 + secp256r1	P-256	+549.2% (6.49×)
Hybrid	ML-KEM-768 + secp384r1	P-384	+379.8% (4.80×)
Hybrid	ML-KEM-1024 + secp521r1	P-521	+510.9% (6.11×)
Hybrid	SMAUG-T1 + secp256r1	P-256	+542.2% (6.42×)
Hybrid	SMAUG-T3 + secp384r1	P-384	+383.2% (4.83×)
Hybrid	SMAUG-T5 + secp521r1	P-521	+509.7% (6.10×)
Hybrid	NTRU+KEM-576 + secp256r1	P-256	+546.9% (6.47×)
PQC-only	ML-KEM-512	P-256	+524.7% (6.25×)
PQC-only	ML-KEM-768	P-384	+360.8% (4.61×)
PQC-only	ML-KEM-1024	P-521	+494.3% (5.94×)
PQC-only	SMAUG-T1	P-256	+524.7% (6.25×)
PQC-only	SMAUG-T3	P-384	+362.5% (4.63×)
PQC-only	SMAUG-T5	P-521	+493.2% (5.93×)
PQC-only	NTRU+KEM-576	P-256	+546.9% (6.47×)

estimate the minimum expected time for full PQC transition. Migration time refers to the period from changing the PQC configuration to sending a SIGHUP to the bridge and waiting until the configuration is applied. For each stage, a

total of 200 migration tests were collected and analyzed through real-time performance monitoring, confirming that PQC migration was successfully executed.

Table 11 presents the performance classification results by stage. To account for the possibility that a new configuration might activate before the bridge settings were fully applied, a 2000 ms delay was introduced during the migration process. The estimated actual migration time (Adjusted Migration Time) was then obtained by subtracting this offset from the measured values.

Each 'prefer' percentage indicates the proportion of classic clients routed to PQC servers. Specifically, 25% means that only one-quarter of classic client sessions are redirected to PQC servers while the rest continue using ECC; 50% corresponds to half of the sessions, and 75% to three-quarters the sessions use PQC servers, and 75% means 75% of sessions use PQC servers.

Therefore, the actual estimated migration time is close to an average of 30 ms, which means that the security policy transition occurs in a seamless and non-disruptive manner with negligible impact on user experience.

Furthermore, the analysis of the IQR during the entire migration process (`Total Migration`) was 13.7 ms, confirming that the policy transitions were carried out stably throughout all stages.

Table 11: Migration times for each PQC migration step according to policy (see Section 3.3.2) are presented together with the results of 200 repeated attempts and the adjusted migration times.

Process	Migration Time (ms)	Adjusted Migration Time (ms)
announce → prefer 25%	2,036.7	36.7
prefer 25% → prefer 50%	2,035.2	35.2
prefer 50% → prefer 75%	2,030.7	30.7
prefer 75% → require	2,027.7	27.7
Full Migration Process	2,032.5	32.5
Total Migration	2032.6	32.6

4.3.2 Minor Update Scenario: Simple Notice Update within the Server

The minor scenario evaluated performance by assuming a situation where announcements are modified during the game start and progression. This process involves a single event rather than a gradual policy transition, and the impact on actual system operation was observed by repeatedly performing minor announcement modifications.

Figure 7 shows textual results confirming that message modifications were properly reflected and displayed during the scenario execution. The average delivery delay time from server application to client exposure was measured to be 40ms.

[INFO] Transform map update complete. New rules:	
Old Value	**New Value**
Welceme to Game Server v2!	**Welcome** to Game Server v2! **[Grammer Fixed]**
Game start	Game start! **[Fixed]**
wins the round!	wins! **[Fixed]**
Errer: It is not your turn.	**Error**: It is not your turn. **[Grammer Fixed]**
Invalid command.	Invalid command… **[Fixed]**
[INFO] Reloading bridge server via SIGHUP...	

Fig. 7: Minor error correction process Log. The old value is replaced with the new value through the SIGHUP signal(See section 3.2.2).

Table 12: Migration times for a minor update within the server are presented together with the estimated actual times. This process corresponds to a minor notification text modification, and the measurement was conducted in the same manner as before, using 200 repeated handshake attempts with a 2000ms delay, following the approach in Table 11.

Server	Migration Time (ms)	Adjusted Migration Time (ms)
v1 (Classic TLS 1.3)	2,042.0	42.0
v2 (PQC)	2,038.3	38.3

5 Conclusion

The advent of quantum computing necessitates a timely and practical migration from classical public-key cryptography to post-quantum standards. Although TLS 1.3 provides an architectural basis for integrating new cryptographic primitives, operational challenges such as larger certificates, increased handshake latency, and legacy-client compatibility hinder direct deployment.

This work proposed a bridge-server-based framework that enables zero-downtime TLS 1.3 migration. By incorporating policy-driven routing, hot reload, and rollback mechanisms, the system realizes practical *crypto agility*, allowing administrators to transition seamlessly between classical and PQC-enabled servers.

Beyond migration, the bridge server also supports operational updates: in *minor updates* (e.g., message corrections), it transparently applies changes without disturbing active sessions, while in *major updates* (e.g., protocol or version changes), multiple bridge instances preserve compatibility and orchestrate phased migration. This dual capability positions the bridge server as a practical step toward sustaining secure TLS services under continuous evolution.

Experimental validation confirmed that the proposed approach preserves service continuity with measurable overhead. In particular, handshake latency was approximately 5.8–6.4× higher than ECC baselines in compute-bound settings, while the relative overhead was substantially smaller in network-bound scenarios—highlighting the importance of deployment context. Despite being conducted in controlled virtualized environments that do not fully capture production-

scale diversity, these experiments establish a reproducible baseline for future evaluations.

Future work includes extending support to a broader range of PQC algorithms, integrating automated monitoring for adaptive policy control, and assessing deployments at larger scale and under diverse network conditions.

References

1. E. Milanov, "The rsa algorithm," *RSA laboratories*, vol. 1, no. 11, 2009.
2. U. M. Maurer and S. Wolf, "The diffie–hellman protocol," *Designs, Codes and Cryptography*, vol. 19, no. 2, pp. 147–171, 2000.
3. P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994.
4. L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
5. N. I. of Standards and Technology, "Selected algorithms — post-quantum cryptography." NIST CSRC, 2024.
6. Korea Post-Quantum Cryptography Research Group, "Official website of korea post-quantum cryptography research group," 2025.
7. E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," Request for Comments RFC 8446, Internet Engineering Task Force, 2018.
8. M. Ivanitskiy, "SSL/TLS Certificate Rotation Without Restarts in NGINX Open Source." NGINX Community Blog, September 2023. Accessed: YYYY-MM-DD.
9. HAProxy Technologies, "Seamless reloads with zero downtime." HAProxy Documentation, 2024. Accessed: 2025-09-19.
10. O. Grote, A. Ahrens, and C. Benavente-Peces, "A review of post-quantum cryptography and crypto-agility strategies," in *2019 International Interdisciplinary PhD Workshop (IIPhDW)*, pp. 115–120, IEEE, 2019.
11. T. Dierks and E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.2," Request for Comments RFC 5246, Internet Engineering Task Force, 2008.
12. "Nist pqc project." https://csrc.nist.gov/Projects/post-quantum-cryptography, 2025. Accessed on 7 June 2025.
13. "Module-lattice-based key-encapsulation mechanism (fips 203)," Tech. Rep. FIPS 203, National Institute of Standards and Technology, 2024. Final publication: Aug 13, 2024.
14. "Module-lattice-based digital signature standard (fips 204)," Tech. Rep. FIPS 204, National Institute of Standards and Technology, 2024. Final publication: Aug 13, 2024.
15. "Stateless hash-based digital signature standard (fips 205)," Tech. Rep. FIPS 205, National Institute of Standards and Technology, 2024. Final publication: Aug 13, 2024.
16. R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber algorithm specifications and supporting documentation," *NIST PQC Round*, vol. 2, no. 4, 2019.
17. D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," in *Proceedings of the 2019 ACM SIGSAC conference on computer and communications security*, pp. 2129–2146, 2019.

18. P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-fourier lattice-based compact signatures over NTRU," *Submission to the NIST's post-quantum cryptography standardization process*, vol. 36, no. 5, 2018.
19. "Nist selects hqc as fifth algorithm for post-quantum encryption." NIST News Release, Mar 2025.
20. J. Kim and J. H. Park, "Ntru+: compact construction of ntru using simple encoding method," *IEEE Transactions on Information Forensics and Security*, 2023.
21. "Smaug-t: The key exchange algorithm based on module-lwe and module-lwr." https://kpqc.or.kr/images/pdf/SMAUG-T_Document.pdf, 2025. Accessed on 7 June 2025.
22. S. Kim, J. Ha, and J. Lee, "Aim: Symmetric primitive for shorter signatures with stronger security," in *ACM CCS 2023: ACM Conference on Computer and Communications Security*, ACM (Association for Computing Machinery), 2023.
23. J. H. Cheon, H. Choe, J. Devevey, T. Güneysu, D. Hong, M. Krausz, G. Land, M. Möller, D. Stehlé, and M. Yi, "Haetae: Shorter lattice-based fiat-shamir signatures," *Cryptology ePrint Archive*, 2023.
24. A. Wiesmaier, N. Alnahawi, T. Grasmeyer, J. Geißler, A. Zeier, P. Bauspieß, and A. Heinemann, "On pqc migration and crypto-agility," *arXiv preprint arXiv:2106.09599*, 2021.
25. "Kpqc competition." https://kpqc.or.kr/competition.html, 2025. Accessed on 7 June 2025.
26. "Performing a rolling update." https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/, 2025. Accessed: 2025-09-20.
27. "Envoy proxy official documentation." https://www.envoyproxy.io/docs/envoy/latest/, 2025. Accessed: 2025-09-20.
28. "Post-quantum cryptography in cloudflare's zero trust platform." https://developers.cloudflare.com/ssl/post-quantum-cryptography/pqc-and-zero-trust/, 2025. Accessed: 2025-09-20.
29. "Openquantumsafe experimental nginx docker image." https://hub.docker.com/r/openquantumsafe/nginx. Accessed: 2025-09-23.
30. N. Koblitz, A. Menezes, and S. Vanstone, "The state of elliptic curve cryptography," *Designs, codes and cryptography*, vol. 19, no. 2, pp. 173–193, 2000.
31. J. H. Cheon, H. Choe, D. Hong, and M. Yi, "Smaug: pushing lattice-based key encapsulation mechanisms to the limits," in *International Conference on Selected Areas in Cryptography*, pp. 127–146, Springer, 2023.
32. A. Tashildar, N. Shah, R. Gala, T. Giri, and P. Chavhan, "Application development using flutter," *International Research Journal of Modernization in Engineering Technology and Science*, vol. 2, no. 8, pp. 1262–1266, 2020.
33. "Hot reload - flutter documentation." https://docs.flutter.dev/tools/hot-reload, 2025. Accessed on 19 September 2025.
34. "signal(7) - linux manual page." https://man7.org/linux/man-pages/man7/signal.7.html, 2025. Accessed: 19 September 2025.
35. Open Quantum Safe Project, "liboqs: C library for quantum-safe cryptographic algorithms." Project website, 2025.
36. OpenSSL Project, *provider(7) — OpenSSL Documentation*, 2025.
37. Open Quantum Safe Project, "oqs-provider: Open quantum safe provider for openssl (3.x)." GitHub repository, 2025.